LLM-Based Architecture Detection in Practice: An Empirical Multi-Model Evaluation

Saveliy Chertkov[1] and Andrey Sadovykh[1]

Innopolis University, Innopolis, Russia
s.chertkov@innopolis.university, a.sadovykh@innopolis.university

Abstract. Knowing the architectural style of a repository — modular monolith, layered, script-based — is a prerequisite for selecting the right quality-improvement tactics, yet classifying hundreds of repositories manually is impractical. This paper reports on what works and what does not when using LLMs for this task. We find that a lightweight, scriptable pipeline based on file trees and import dependency graphs achieves 70.2% accuracy across 57 manually labeled repositories — good enough for automated triage, with no GPU or fine-tuning required. We also identify two critical failure modes: (1) all five evaluated models systematically confuse modular monolith and layered architectures, a confusion that does not diminish with richer input; and (2) model confidence scores are uncalibrated and provide no reliable signal for filtering uncertain predictions. Code signatures, despite being intuitive to include, actively degrade performance and should be omitted. Both findings have direct implications for practitioners building architecture-aware automation pipelines.

Keywords: software architecture detection · large language models · architectural classification · repository analysis · empirical software engineering

1 Introduction

Large organizations routinely need to assess the architectural style of dozens or hundreds of repositories for technical debt assessment, migration planning, or tooling configuration — yet manual classification requires significant expertise and does not scale [3]. A reliable automated first step would enable downstream tools to select and apply appropriate quality-improvement tactics [2] [8].

Static analysis tools can extract structural metrics but lack semantic reasoning sufficient to distinguish, for example, a modular monolith from a layered architecture [6]. While recent LLMs show strong code-understanding capabilities [7] [10], no systematic evaluation of repository-level architectural classification exists. This paper reports on five models and four evidence configurations, addressing:

- **RQ1:** How accurately can LLMs classify the architectural style of a Python backend repository?

- **RQ2:** Which types of repository-level evidence contribute most to classification accuracy?
- **RQ3:** How do different LLMs compare in architectural classification performance?

Our contributions are: (1) a manually labeled dataset of 57 Python repositories across three architectural styles; (2) a systematic comparison of four context configurations across five LLMs showing import graphs are the most valuable evidence; and (3) evidence that all five models confuse modular monolith and layered architectures, with direct practical implications for tool designers. This work is the first stage of a broader architecture-aware pipeline; tactic selection and implementation are under active investigation.

2 Background

Software architecture and quality. Architectural styles constrain how components interact and directly shape ISO/IEC 25010 maintainability subcharacteristics — modularity, analysability, and modifiability [2] [1]. Márquez et al. [8] systematically mapped architectural tactics to quality attributes, and Kim et al. [5] showed that selecting appropriate tactics requires knowing the system's style. This coupling between style identification and downstream quality improvement is the direct motivation for our work.

Architecture recovery and classification. Traditional architecture recovery reconstructs detailed architectural models from source artifacts [11]. Such approaches are expensive and require expert interpretation. A lighter task — coarse-grained *style classification* — is sufficient for automation: knowing that a system is modular monolith vs. layered vs. script-based enables appropriate tactic selection without full model reconstruction. Static analysis tools can extract structural metrics but lack the semantic reasoning needed to distinguish styles that share similar package structures [6]. To our knowledge, no prior work has systematically evaluated automated style classification at repository scale.

LLMs in software engineering. LLMs demonstrate strong code-understanding capabilities [7], and Piao et al. [10] found that prompts enriched with architectural context improve output quality, directly motivating our evidence-type investigation. However, Horikawa et al. [4] showed that even agentic LLM approaches produce mostly low-level edits with limited high-level design changes, leaving repository-level reasoning an open question.

Research gap. Esposito et al. [3] surveyed generative AI for software architecture and explicitly identified repository-level style classification as a key open area. Our paper is the first controlled multi-model empirical evaluation providing baselines across five models and four evidence configurations.

3 Methodology

3.1 Architectural Taxonomy

We define three styles applicable to Python backend repositories. **Script-based**: few script files, flat structure, limited decomposition. **Layered**: API/service/data layers with directional dependencies. **Modular monolith**: feature- or domain-organized packages without a strict layer hierarchy. Microservices and event-driven styles were excluded because they are distinguishable at the deployment level (multiple independently deployable services), not from single-repository structural evidence. The MM/layered distinction is maintained because the two styles imply different decomposition strategies with distinct tactic implications [2].

3.2 Dataset

Repositories were sourced from GitHub via the Search API. Python was chosen for its dominant role in open-source backend development and its consistent AST-based extraction toolchain. Inclusion criteria: Python source files and `requirements.txt`, no ML-heavy projects (Jupyter Notebooks or dominant data-science libraries), evidence of active development (stars: 10–28,230). After filtering, 59 repositories were retained; one failed artifact extraction due to size, leaving **57 repositories** (33 modular monolith, 16 layered, 8 script-based; 5–312 Python files, median 38; 3–847 import edges, median 61). Each was cloned at a fixed commit hash. Ground truth labels were assigned by a single expert annotator following the taxonomy in Section 3.1.

3.3 Repository Evidence Extraction

For each repository: (1) **file tree** — ASCII directory structure; (2) **import dependency graph** — internal module import relationships via AST parsing; (3) **code signatures** — class definitions and function signatures via AST (no bodies); (4) **structural metrics** — module count, average fan-out, max fan-in, cycle detection, edge count. Artifacts were passed to models without truncation; all 57 repositories completed within the 300-second timeout across all configurations.

3.4 Prompt Design

Four incremental context configurations: **P1** (file tree only); **P2** (+import graph); **P3** (+code signatures); **P4** (+structural metrics). All prompts share an identical system prompt specifying the task, taxonomy, allowed labels, and required JSON output schema.

3.5 Models

Five LLMs accessed via the Ollama API (temperature = 0.2, timeout = 300s, retries = 2, one run per repository per model): Qwen3-coder-next, DeepSeek-v3.2, Gemini-3-Flash-Preview, Gemma4-31B, MiniMax-M2.7 (all cloud variants; abbreviated in tables as Qwen3, DeepSeek, Gemini, Gemma4, MiniMax).

4 Results

Table 1 shows accuracy and macro-F1 across all 20 model–prompt combinations. The majority-class baseline (predicting modular monolith always) yields 57.9%; the best configuration exceeds this by 12.3 percentage points. The most important pattern is not which model wins but how dramatically accuracy shifts across prompt configurations: the evidence-type effect is consistent across all five models, while adjacent model rankings on N=57 differ by one or two predictions and should be treated as indicative rather than definitive.

4.1 Overall Classification Performance (RQ1)

Table 1. Accuracy and macro-F1 across models and prompts. Best result per column in bold.

Model	P1		P2		P3		P4	
	Acc	$F1_m$	Acc	$F1_m$	Acc	$F1_m$	Acc	$F1_m$
Qwen3	**.596**	.53	**.702**	**.65**	.632	.52	**.667**	**.57**
MiniMax	.579	**.55**	.649	.61	**.649**	**.55**	.649	.56
Gemma4	.561	.48	.632	.55	.614	.47	.614	.45
DeepSeek	.544	.46	.614	.50	.614	.48	.632	.49
Gemini	.526	.47	.579	.49	.561	.41	.596	.45

Best result: Qwen3 at P2 — 70.2% accuracy, 0.65 macro-F1.

4.2 Impact of Evidence Types (RQ2)

The accuracy delta from each evidence addition is readable from Table 1. Adding the import graph (P1→P2) improves accuracy for every model, with gains from +5.3% (Gemini) to +10.5% (Qwen3). Adding code signatures (P2→P3) reduces or does not change accuracy for every model; Qwen3 drops 7 percentage points. Structural metrics (P3→P4) provide marginal inconsistent gains. Notably, import graph extraction is pure AST parsing with negligible overhead, while code signatures add substantial context volume with negative effect — an unambiguous design choice.

Table 2. Per-class accuracy at P2 (file tree + import graph).

Class	Qwen3	MiniMax	Gemma4	DeepSeek	Gemini
Modular monolith (33)	**.727**	.636	.606	.515	.424
Layered (16)	.750	.688	.750	.750	.750
Script-based (8)	.750	.750	.750	**.875**	.750

4.3 Per-Class Performance

Script-based and layered are reliably detected (≥0.69 across all models); modular monolith is the primary differentiator and should be treated with lower confidence.

4.4 Model Comparison (RQ3)

Qwen3 achieves the highest accuracy (.702) and benefits most from import graphs (+10.5%). MiniMax produces the smallest correct/incorrect confidence gap (Section 4.6) — the safer choice when confidence scores are consumed downstream. DeepSeek leads on script-based detection (.875 at P2; Table 2). Gemini has the lowest overall accuracy despite reporting the highest prediction confidence, a calibration inversion that motivates Section 4.6.

4.5 Systematic Misclassification Patterns

Table 3. Modular monolith misclassifications at P2 (N = 33).

Model	Correct	→layered	→script	→other
Qwen3	**24**	8	1	0
MiniMax	21	7	2	3
Gemma4	20	9	3	1
DeepSeek	17	11	5	0
Gemini	14	14	4	1

4.6 Confidence Calibration

No model discriminates correct from incorrect predictions by confidence ($\Delta \leq 0.026$, all models); all five report mean confidence ≥ 0.82 regardless of correctness. Confidence scores provide no useful signal for filtering uncertain predictions.

Table 4. Mean confidence: correct vs. incorrect predictions (P2).

Model	Correct	Incorrect	Δ
MiniMax	.838	.812	**.026**
Qwen3	.893	.876	.017
Gemma4	.918	.904	.014
DeepSeek	.853	.839	.014
Gemini	.916	.907	.009

5 Discussion

The evidence-type results have a clear mechanistic explanation. Import graphs encode inter-module coupling — the structural signal that directly reflects architectural decomposition decisions [9] [2] — while file trees capture only naming hierarchy. Code signatures introduce a large volume of low-level naming information that dilutes the dependency signal without adding structural content. The import graph is both the most informative and the cheapest artifact to produce.

5.1 The Modular Monolith / Layered Boundary

The systematic MM/layered confusion (Table 3) is the study's most important negative finding. Both styles produce similar structural evidence: multi-package codebases with a web of internal imports. The distinguishing criterion — whether packages represent technical layers (presentation, service, data) or domain features (users, orders, notifications) — is a semantic property of module naming, not a structural property visible in import topology. A file tree and import graph alone are insufficient to resolve it; explicit naming-convention guidance or module-name semantic analysis may be necessary to close this gap. The consistent mis-assignment to "layered" suggests a training-data prior toward the more frequently documented style.

5.2 Confidence Miscalibration

The near-zero Correct/Incorrect Δ in Table 4 means LLMs cannot introspect on their own architectural reasoning: a wrong answer carries the same confidence as a correct one. Gemini is the clearest case — highest mean confidence (.913), lowest accuracy (.579 at P2). The operational implication is clear: confidence thresholds cannot be used to filter uncertain predictions, and all MM/layered cases should be routed to human review regardless of reported confidence.

5.3 Practitioner Guidance

Three recommendations follow directly from the results. (1) **P2 (file tree + import graph) is the correct default.** Adding code signatures degrades accuracy and should be avoided; structural metrics provide no consistent benefit.

Import graph extraction is pure AST parsing — lightweight, scriptable, no LLM calls required. (2) **Disregard confidence scores entirely.** No model distinguishes correct from incorrect predictions by confidence; high confidence is the default regardless of correctness. (3) **The MM/layered boundary is unreliable.** When the distinction matters downstream, route all MM/layered predictions to human review; MM→layered misclassification is the costlier direction, as it applies layer-specific tactics to a domain-organized codebase. When the distinction is not required, collapse to a two-class taxonomy (script-based vs. non-script) for substantially higher accuracy.

6 Threats to Validity

Internal: Ground truth was assigned by a single annotator — the most significant validity threat. The MM/layered boundary, which is the central finding of the paper, is also the hardest annotation call: some repositories genuinely straddle the two styles and a second annotator may draw the line differently. We mitigated this by defining the taxonomy before labeling, but inter-annotator agreement was not measured. Each prompt was executed once per model; temperature = 0.2 reduces but does not eliminate non-determinism. Prompt templates were finalized before evaluation runs, though taxonomy awareness was shared between prompt development and labeling. Model identifiers are Ollama API tags without pinned snapshot hashes, limiting exact reproducibility. **External:** Results are limited to 57 Python backend repositories; generalization to other languages, domains, or a broader taxonomy requires further study. **Construct:** Some repositories exhibit hybrid characteristics; MM/layered confusion in LLM outputs may partly reflect genuine ground-truth ambiguity rather than model error alone. **Conclusion:** The dataset is small and class-imbalanced; all rankings and deltas are preliminary baselines, not definitive performance claims.

7 Conclusion

This study yields three findings for practitioners and tool builders. First, 70.2% accuracy (Qwen3, file tree + import graph) is achievable with a lightweight, fully scriptable pipeline — no GPU, no fine-tuning — sufficient for triage at repository-portfolio scale. Second, import graphs are the essential evidence; code signatures are harmful. Third, modular monolith/layered confusion and systematically miscalibrated confidence scores are fundamental limitations shared by all five evaluated models, not artefacts of a single architecture.

Closing the gap toward 85%+ accuracy will require multi-annotator ground truth on a larger dataset and a hybrid approach combining LLM classification with static structural thresholds — directions we are pursuing as part of a broader architecture-aware quality improvement pipeline that uses style detection as the trigger for tactic selection and implementation.

Data Availability

The labeled dataset, repository artifacts, prompt templates, raw model outputs, and evaluation scripts are available at: `https://github.com/anonymous-ghub/Architecture-Detection-in-Software-Repositories`

Acknowledgements

AI tools were used for: writing assistance (GitHub Copilot, Claude Sonnet); code generation (GitHub Copilot); data analysis and table generation (Claude Sonnet); literature discovery (Semantic Scholar MCP, NotebookLM). Conceptualization, methodology, data labeling, and scientific conclusions are the authors' own.

References

1. ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model. `https://iso25000.com/en/iso-25000-standards/iso-25010` (2023), [Online; accessed 2025-12-04]
2. Bass, L., Clements, P., Kazman, R.: Software Architecture in Practice. Addison-Wesley, 4th edn. (2021)
3. Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., Taibi, D.: Generative ai for software architecture. applications, trends, challenges, and future directions. Applications, Trends, Challenges, and Future Directions (2025)
4. Horikawa, K., Li, H., Kashiwa, Y., Adams, B., Iida, H., Hassan, A.E.: Agentic refactoring: An empirical study of AI coding agents. In: arXiv preprint arXiv:2511.04824 (2025)
5. Kim, S., Kim, D.K., Lu, L., Park, S.: Quality-driven architecture development using architectural tactics **82**(8), 1211–1231 (2009)
6. Lenarduzzi, V., Pecorelli, F., Saarimaki, N., Lujan, S., Palomba, F.: A critical comparison on six static analysis tools: Detection, agreement, and precision **198**, 111575 (2023)
7. Liu, B., Jiang, Y., Zhang, Y., Niu, N., Li, G., Liu, H.: Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. Automated Software Engineering **32**(1) (2025). `https://doi.org/10.1007/s10515-025-00500-0`
8. Márquez, G., Astudillo, H., Kazman, R.: Architectural tactics in software architecture: A systematic mapping study. Journal of Systems and Software **197**, 111558 (2022). `https://doi.org/10.1016/j.jss.2022.111558`
9. Perry, D.E., Wolf, A.L.: Foundations for the study of software architecture **17**(4), 40–52 (1992). `https://doi.org/10.1145/141874.141884`
10. Piao, Y.C.K., Paul, J.C., Da Silva, L., Dakhel, A.M., Hamdaqa, M., Khomh, F.: Refactoring with LLMs: Bridging human expertise and machine understanding. In: arXiv preprint arXiv:2510.03914 (2025)
11. Shokri, S.E., Khatchadourian, R.: IPSynth: Interprocedural program synthesis for software architecture recovery and architectural tactics detection. In: arXiv preprint arXiv:2403.10836 (2024)